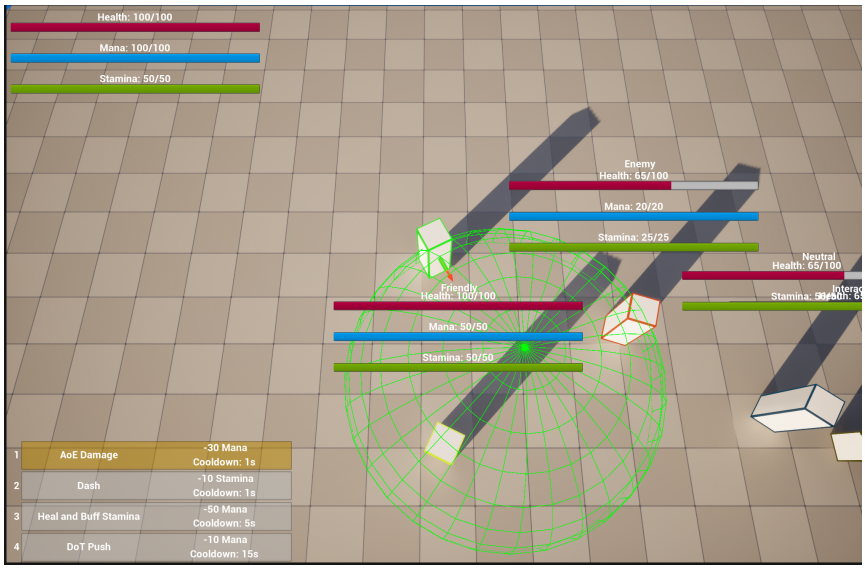
Programming for Game Engines Assignment 2

A Modular Ability System Implementation in Unreal Engine 5

Oliver J. Martin
MSc Computer Games



The ability system running in Unreal Engine 5.

Department of Computing
Goldsmiths, University of London

April 23, 2026

Links

Video: `https://youtu.be/cIZZLWoxqow`
GitHub Repository: `https://github.com/OdgeM/GameEnginesDemoProject`

1 Introduction

This project implements a modular ability and stat system inspired by action RPGs. The aim was to design a flexible framework where gameplay effects (i.e. damage, buffs, movement) can be defined independently and applied to different actors with varying capabilities. To achieve this, the system separates the data and behaviour of each component: thus, allowing for reusable and extensible gameplay logic. It draws from common practice in modern game development - particularly component-based architecture and effect-driven systems. While the bulk of the core logic was implemented in C++, the goal was to allow for high-level customisation of abilities and stats from derived blueprints - therefore allowing less-technical designers to work with the system quickly and easily, without having to trawl through the source code.

2 Program Overview

The system allows actors to:

- Possess a configurable set of stats (for the purposes of this demonstration, these are health, mana, and stamina)
- Execute abilities composed of modular effects
- Target abilities with a mouse-based system, while previewing the ability
- Apply ability outcomes on click

3 Implementation

3.1 Stat System

Stats are managed and modified for each actor within the `UStatComponent` class. Within this component, stats are stored within a `TMap<EStatsType, FStatData>`, where `EStatsType` is an enumerator which stores the possible stat types and `FStatData` is a struct which contains the current value and maximum value of each stat. Access to the stats is handled via Blueprint-callable functions, which follow a "bool and out" parameter pattern: whereby the function returns a bool if the operation was a success, and a parameter that is passed by reference returns the result - with the aim of safely handling missing stats.

3.2 Stat Modifiers

Modifications to stats utilise the `FStatModifier` struct. This struct contains:

- The target stat, as an `EStatsType` value

- The operation to apply to the stat, defined by the `EModifierOp` enum - which contains "Add", "Multiply" and "Override" options
- The magnitude of the operation to apply to the stat's current value, as a float
- The magnitude of the operation to apply to the stat's maximum value, as a float
- Two Boolean behaviour flags:
  - `bOverTime`: Whether the change should be applied over time
  - `bTemporary`: Whether this is a temporary change that should be reverted
- And a float which defines the duration of the effect if either of the above Booleans are true.

Stat changes are then centralised in the `UStatComponent::ApplyModifier` function, which ensures that there is only a single source of truth for the stat logic. Effects over time and temporary effects are handled using timers, which allows for multiple overlapping effects.

3.3 Regeneration

To have the actor passively regenerate stats for features like automatic health or mana regeneration, the component also contains a `RegenRate` member - which is a `TMap<EStatsType, float>` containing how much of that stat should be regenerated per second.

3.4 Abilities

Abilities are managed by the `UAbilityComponent` class, which stores an array of the current abilities and handles the logic for adding, selecting, targeting, and activating those abilities. Abilities are represented by the `UAbility` class which contains the following Blueprint-editable member variables, which can be used to configure the ability:

- `FString Name`: The name of the ability to be used in the UI.
- `TArray<UAbilityEffect*> Effects`: The effects of the ability.
- `UAbilityTargeting* TargetingStrategy`: How the ability is targeted.
- `int32 TargetableTeams`: The "teams" that the ability can target (i.e. allies/enemies etc.)
- `float Cooldown`: How long it takes for the ability to cooldown.
- `TArray<UAbilityEffect*> Costs`: The stat costs required to activate the ability.

3.5 Ability Effects

The `UAbilityEffect` class is used to define the effects that an ability has. The main `UAbilityEffect` just contains an empty `Apply` function, which is overridden by subclasses to define custom logic. This allows for custom logic to be implemented as needed, without requiring any changes to the overall system structure. For this demonstration, three example effects were implemented:

- `UStatEffect`: Applies an array of `FStatModifiers` to the target actors which is used for any effect that requires stat changes (i.e. damage, healing, buffs etc.).
- `UPushEffect`: Applies a force to the target actors to create a knockback effect.
- `UDashEffect`: Launches the instigating actor forwards.

3.6 Ability Targeting

Like effects, the targeting of abilities is handled by modular `UAbilityTargeting` classes which contains a `GetTargets` function which defines the targeting logic and an `UpdatePreview` function to update the visual effects previewing the ability in the game world. As with the effects, separating the targeting from the rest of the ability allows for high levels of extensibility and customisability. For this demonstration, the following targeting methods were created:

- `USingleTargeting`: A single actor is targeted.
- `UAoETargeting`: All actors within a sphere are targeted.
- `UConeTargeting`: All actors within a cone in front of the instigating actor are targeted.
- `ULineTargeting`: All actors within a line in front of the instigating actor are targeted.

Targeting data is contained within the `FAbilityTargetingData` struct. This struct contains:

- `AActor* HoverActor`: The actor currently underneath the mouse.
- `TArray<AActor*>`: The actors that are targeted by the ability.
- `FVector TargetLocation`: The world location that is being targeted.
- `FVector SourceLocation`: The world location where the effect will originate from.

The targeting data is passed by reference to the `GetTargets` function, which modifies the members as defined by its logic. This is then passed to the `UAbilityEffect` belonging to the ability, to define where the effect is applied.

3.7 Teams and the Targetable Interface

To allow different abilities to target different types of actors, an `ETeam` enum is used to define the possible "teams" that the actors belong to. For this demonstration these are:

- Player
- Enemy
- Neutral
- Friendly
- Interactable

To allow an ability to target combinations of these, and to make this easy to use within Blueprints, this is used as a bitmask within the `UAbility` class - which creates a nice checkbox list within the editor.

To define which actors are targetable, a "Targetable" interface was implemented with a function to get the owner's team and to set the owner as "targeted". During targeting, actors are filtered by this interface, and the `UAbility` then further filters which actors belong to the targeted teams.

3.8 Previewing Abilities

Each `UAbilityTargeting` class contains logic to display an ability preview in the world - such as drawing a sphere for `UAoETargeting`. To further aid with targeting, the level contains a post-processing volume with a custom material which adds an outline to actors while targeting - with the colour representing their team (or black when not targetable), and the outline glowing brighter when that actor is targeted.

3.9 Input System

A `UPlayerController` subclass was created to handle the inputs for ability selection and activation. Using InputActions, abilities can be selected with the number keys which starts targeting. While targeting, the player controller casts a raycast at current mouse position, using the hit location and hit actor to constructs a `FAbilityTargetingData` struct - which is then passed by reference to the ability to allow the targeting strategy to populate the target actors and display the preview.

While targeting, left click activates the current ability, pressing a different number key changes the current ability, and pressing the same number key again deselects the current ability. If left click is pressed while no ability is selected, the character moves to the clicked position.

3.10 UI

A minimal UI was designed to demonstrate that the widget blueprints can interact with the C++ classes, retrieving stats and ability information.

4 Problem Solving

4.1 Stacking Effects

For effects applied over time, stacking multiple effects with different durations and magnitudes proved difficult. As mentioned, this was handled by using timers, which apply repeated modifiers over time and then automatically clear when the duration expires. This required that `FTimerHandle` for each effect had to be stored and identifiable - even after they were out of scope as otherwise, they would never be cleared. To solve this, `TSharedRef` shared handles were used to provide dynamic pointers to the handle which can then be captured in the timer lambda function.

4.2 Outline Material

As the post-processing material that applies the actor outlines during targeting was in a volume covering the whole play space, changing the outline colour of individual actors based on their team

and whether they are targeted was a challenge. Simply modifying a parameter on the material would apply the change to all actors so, instead, custom depth stencils were used to encode the information.

Custom depth was already being used within the material to define the areas to apply outlines to, so using stencil values proved to be a natural extension. While the stencil values could be hard coded for each team used in this demonstration, this lacks the extensibility that this system is aiming for. Instead, the actor's team was used as the stencil value and, as each actor can only belong to one team, the colour could be selected based on that - with the stencil value being equal to 2 raised to the power of the team's position in the `ETeam` enum. However, to enable the outline to glow brighter when targeted, this proved insufficient.

To encode the targeting information, the bit-wise representation of the `ETeam` value was considered. In this example, the `ETeam` value was represented in the five lowest bits of the 32-bit integer - with each bit representing a different team. For example, the value for "friendly" would be:

0 1 0 0 0

Interactable Friendly Neutral Enemy Player

As the additional information was just a Boolean value, only a single additional bit would be required. Therefore, the team value was bit-shifted left, and the new rightmost bit was used as a flag indicating if the actor was being targeted. In integer arithmetic, this process is equivalent to multiplying the integer team value by two and adding one if the actor is being targeted.

Then, using this as the stencil value, the material can then retrieve the original team value by dividing the stencil value by two and taking the floor of the new value - which is then used to select the correct colour. The targeting status is then decoded by taking checking if the remainder of the stencil value division is 0.5 or 0. If it is 0.5, then the bit flag was enabled and the actor is being targeted and, therefore, the brightness of the outline is increased.

5 Future Work

Several potential extensions could significantly improve both the flexibility and robustness of this system.

5.1 Advanced Effect Stacking and Management

The current system enables multiple effects to run simultaneously but lacks any formal stacking rules. Future work could introduce:

- Stacking policies (e.g. refresh duration, accumulate intensity, or replace existing effects)
- Effect identification IDs to track and manage individual instances
- Priority systems for conflicting modifiers

This would ensure that the effects interact predictably.

5.2 Data-Driven Ability Design

Currently, abilities and effects are defined in code or Blueprint classes. This could be improved by introducing:

- DataAssets or data tables to define abilities externally
- Designer-friendly configuration without recompilation
- Easier balancing and iteration

This would improve the scalability greatly, as it would further separate game logic from content.

5.3 Visual Effects and UI Improvements

The current UI is extremely barebones, and abilities lack visual effects altogether. Adding the option to configure abilities with casting and impact VFX and UI details (such as icons) would greatly improve the system.